

PHASE 3

DATABASES

High-End System Design Interview Notes

THE BIG IDEA

A database is not just where data is stored. In system design, the database choice controls how data is modeled, queried, protected, scaled, and recovered.

SQL → Transactions → Concurrency → Optimization → NoSQL → Trade-offs

Phase 3 Learning Map

1. SQL foundations: tables, relationships, primary/foreign keys
2. Indexes and query performance
3. Joins and relational querying
4. Transactions + ACID
5. Isolation levels + locks + concurrency
6. Query optimization
7. NoSQL: document, key-value, and wide-column databases
8. Access-pattern-first design
9. Consistency and operational trade-offs
10. Interview question: Why SQL instead of NoSQL?

INTERVIEW MINDSET

Do not answer database questions with “SQL is better” or “NoSQL scales better.” The correct answer depends on data relationships, consistency requirements, access patterns, scale, and operational constraints.

1. Tables, Relationships, Primary & Foreign Keys

A relational database stores structured data in tables. Each table represents an entity or concept, while relationships connect entities.

Table

- Rows = records / instances.
- Columns = attributes / properties.
- A schema defines the structure and data types.

```
CREATE TABLE users (  
  user_id INT PRIMARY KEY,  
  name VARCHAR(100),  
  email VARCHAR(255) UNIQUE  
);
```

Primary Key

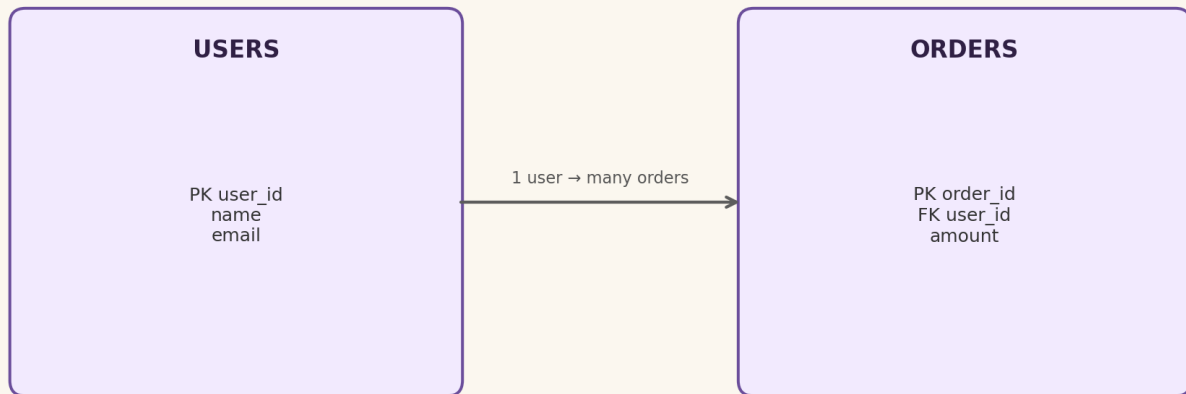
- Uniquely identifies each row.
- Should be stable and non-null.
- Examples: user_id, order_id, product_id.

Foreign Key

- Stores a reference to a row in another table.
- Enforces a relationship between tables.
- Helps maintain referential integrity.

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  user_id INT NOT NULL,  
  amount DECIMAL(10,2),  
  FOREIGN KEY (user_id) REFERENCES users(user_id)  
);
```

Relational Database: Tables + Relationships



Foreign key connects related rows without duplicating the parent identity.

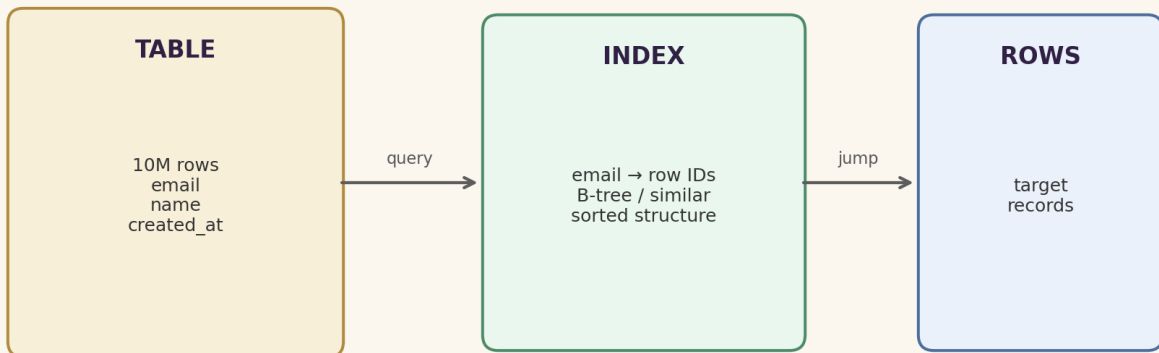
INTERVIEW TIP

A foreign key answers: "Which parent record does this child record belong to?" Example: `orders.user_id` points to `users.user_id`.

2. Indexes

An index is an additional data structure that helps the database find rows faster, similar to an index in a textbook.

Index: Faster Lookup, Extra Storage



⚡ Why indexes help

- Without a useful index, the database may scan many rows.
- With a suitable index, it can locate candidate rows much faster.
- Commonly indexed columns: primary keys, foreign keys, frequent filter columns, join keys, and some sort columns.

⚠ The trade-off

- Indexes consume storage.
- INSERT/UPDATE/DELETE can become more expensive because indexes must be maintained.
- Too many indexes can hurt write-heavy systems.

```

CREATE INDEX idx_users_email
ON users(email);

SELECT * FROM users
WHERE email = 'alice@example.com';
  
```

KEY IDEA

Index design should follow real query patterns. “Index everything” is not a good production strategy.

3. Joins

A JOIN combines rows from multiple tables using related columns.

Main JOIN types

JOIN	Meaning	Typical use
INNER JOIN	Only matching rows	Users who have orders
LEFT JOIN	All left rows + matches	All users, even without orders
RIGHT JOIN	All right rows + matches	Less common; mirror of LEFT
FULL OUTER JOIN	All rows from both sides	Compare complete sets

```
SELECT u.name, o.order_id, o.amount
FROM users u
INNER JOIN orders o
ON u.user_id = o.user_id;
```

SYSTEM DESIGN CONNECTION

Relationships are one reason SQL is attractive: the database can represent and query related entities directly instead of forcing the application to manually assemble everything.

4. Transactions

A transaction groups multiple database operations into one logical unit of work.

```
BEGIN;

UPDATE accounts
SET balance = balance - 500
WHERE account_id = 1;

UPDATE accounts
SET balance = balance + 500
WHERE account_id = 2;

COMMIT;
```

COMMIT vs ROLLBACK

- COMMIT → make the transaction's changes permanent.
- ROLLBACK → undo the uncommitted changes when the transaction cannot safely complete.

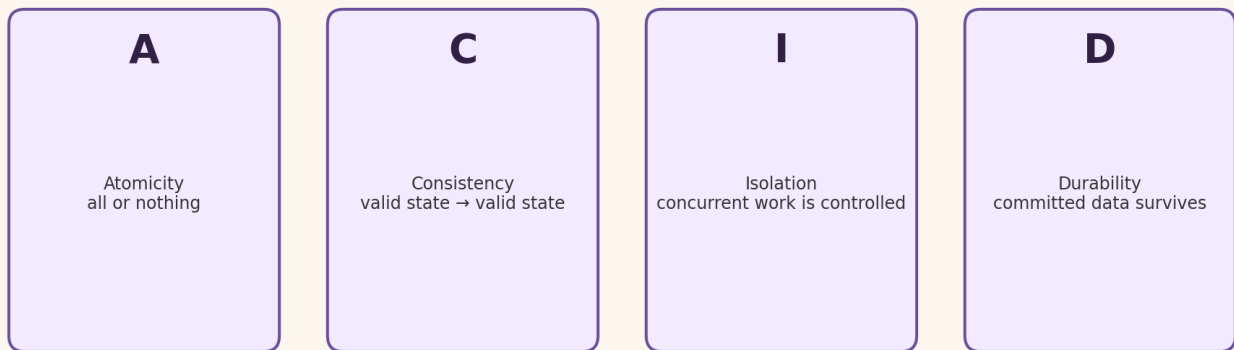
REAL EXAMPLE

Bank transfer: debit A and credit B should be treated as one logical operation. If the credit fails, the system should not leave only the debit applied.

5. ACID

ACID describes important transaction guarantees provided by many relational database systems.

ACID: Reliable Transactions



- Atomicity — all operations in the transaction happen, or none of them do.
- Consistency — a successful transaction leaves the database satisfying its defined rules and constraints.
- Isolation — concurrent transactions are controlled so their intermediate work does not incorrectly interfere.
- Durability — once a transaction is committed, its result is designed to survive failures.

MEMORY TRICK

A-C-I-D = All-or-nothing • Correct state • Concurrent safety • Data survives

6. Isolation Levels

When multiple transactions run at the same time, isolation controls how much one transaction can observe from another.

Isolation Levels: More Protection vs More Concurrency



Problem	Simple meaning	Typical concern
Dirty read	Read data another transaction has not committed	May observe data that later disappears
Non-repeatable read	Same row returns a different committed value later	Concurrent update changed it
Phantom read	Repeated query returns extra/missing matching rows	Concurrent insert/delete affects result

INTERVIEW TIP

Do not memorize isolation levels only as names. Explain the anomalies they prevent and the concurrency cost of stronger isolation.

7. Locks & Concurrency

Concurrency means multiple users or transactions access data at the same time. Locks are one mechanism databases use to coordinate conflicting operations.

Common concepts

- Shared/read lock — permits compatible reads while restricting conflicting changes, depending on the database and isolation mode.
- Exclusive/write lock — protects data being changed from conflicting operations.
- Row-level locking — limits contention to particular rows when supported.
- Deadlock — two or more transactions wait on each other in a cycle.

✦ Deadlock example

- Transaction A locks Row 1 and waits for Row 2.
- Transaction B locks Row 2 and waits for Row 1.
- Neither can continue.

HOW SYSTEMS HANDLE IT

Databases can detect deadlocks and abort one transaction so the other can proceed. Applications should be prepared to retry safely where appropriate.

8. Query Optimization

Query optimization means reducing unnecessary work so a query returns the required result efficiently.

🔍 What to inspect

- Indexes — can the database use an appropriate index?
- Query shape — are you filtering early and returning only needed columns?
- JOINS — are join keys indexed and are unnecessary joins avoided?
- Data volume — how many rows are scanned?
- Execution plan — what does the optimizer actually choose?
- Pagination — avoid inefficient patterns for very large result sets.

```
-- Prefer only required columns
SELECT id, name
FROM users
WHERE email = 'alice@example.com';

-- Inspect the database's chosen plan
EXPLAIN SELECT id, name
FROM users
WHERE email = 'alice@example.com';
```

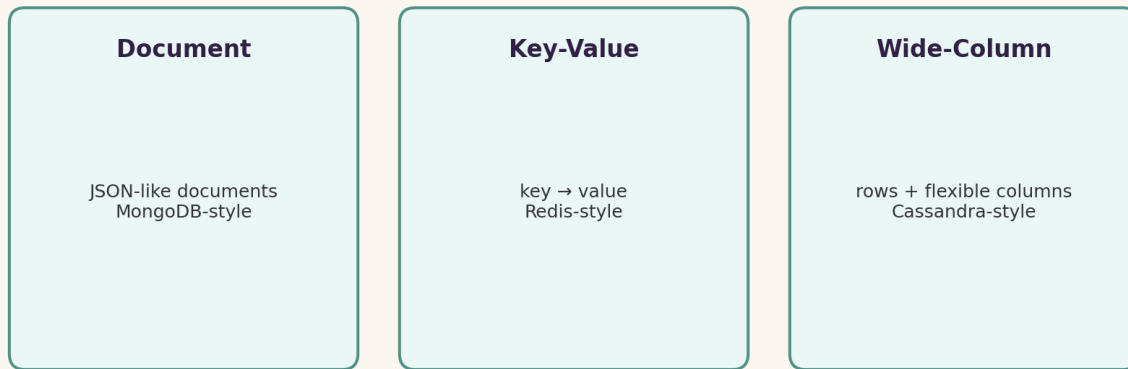
IMPORTANT

Do not optimize by guessing. Measure the query, inspect its execution plan, identify the bottleneck, then change the design and measure again.

9. NoSQL — The Big Picture

NoSQL is a broad family of databases that use models other than the classic relational-table approach. The goal is not simply “more scale”; different NoSQL systems are optimized for different data and access patterns.

Common NoSQL Families



9.1 📄 Document Stores

- Store records as document-like structures, often JSON/BSON-like.
- Documents can contain nested objects and arrays.
- Useful when application data naturally maps to a document and related data is frequently read together.
- Example use cases: product catalogs, user profiles, content systems.

9.2 ⚡ Key-Value Stores

- Data is addressed by a key: key → value.
- Very simple access model can provide fast lookups.
- Useful for sessions, caching, counters, feature flags, and other direct-key access patterns.

9.3 🏠 Wide-Column Stores

- Designed around partitioned rows and flexible columns rather than fixed relational tables.
- Often suited to very large distributed datasets and predictable partition-key-based access.
- Useful when high write/read scale and known query patterns dominate the design.

10. Access-Pattern-First Design

THE SHIFT IN THINKING

SQL often starts with entities and relationships. Many NoSQL designs start with the questions the application must answer: “What will I read, write, update, and at what scale?”

Example: Social Feed

- Requirement: fetch the latest posts for a user's feed.
- Think about the access pattern: “get feed for user X, newest first.”
- A NoSQL design may store data in a shape optimized for that read rather than normalize every relationship.
- Denormalization can reduce joins, but it creates duplication and update complexity.

RULE

Design the data model around the most important access patterns, while accounting for consistency, write amplification, storage, and operational complexity.

11. Consistency Trade-offs

Distributed databases often require choices about how quickly replicas converge and how strongly reads must reflect the latest write.

Choice	What it means	Example
Strong consistency	Reads are constrained to reflect the latest committed state according to the system's guarantees	Account balance / critical inventory
Eventual consistency	Replicas may temporarily differ, but converge when updates propagate	Some feeds, counters, analytics
Availability / latency	Keeping responses available and fast can sometimes require tolerating stale data	Global distributed services

CAREFUL

“Eventual consistency” does not mean “data is lost.” It means replicas can temporarily disagree before convergence, depending on the system's guarantees.

12. When NoSQL Is Useful

- Very large distributed workloads where the chosen NoSQL model fits the access pattern.
- Flexible or rapidly evolving document structures.
- Extremely high-throughput key-based access.
- Workloads where denormalized data can avoid expensive joins.

- Globally distributed applications where the database's replication/partitioning model fits the requirements.
- Systems with predictable access patterns that can be modeled directly.

WHEN SQL MAY FIT BETTER

Strong relationships, complex joins, rich ad-hoc queries, transactional workflows, mature constraints, and strong consistency requirements can make a relational database a natural choice.

13. SQL vs NoSQL — Decision Framework

Factor	SQL	NoSQL
Relationships	Native relational model + joins + constraints	Often application/data-model specific; may embed or duplicate data
Consistency	Strong transactional guarantees are common	Varies widely by product and configuration
Access patterns	Flexible querying is often a strength	Often optimized around known access patterns
Schema	Structured schema	Often more flexible, depending on model
Scale	Can scale vertically and horizontally depending on engine/design	Many systems emphasize distributed horizontal scaling
Operations	Mature tooling and familiar relational operations	Operational model varies significantly by database
Best fit	Relational, transactional, query-rich workloads	Specific high-scale/flexible/distributed access patterns

DO NOT USE A SLOGAN

“SQL for small systems, NoSQL for big systems” is an oversimplification. Large systems can use SQL, NoSQL, or both.

14. Interview Question: Why SQL Instead of NoSQL?

A strong answer should compare requirements rather than declaring one database universally better.

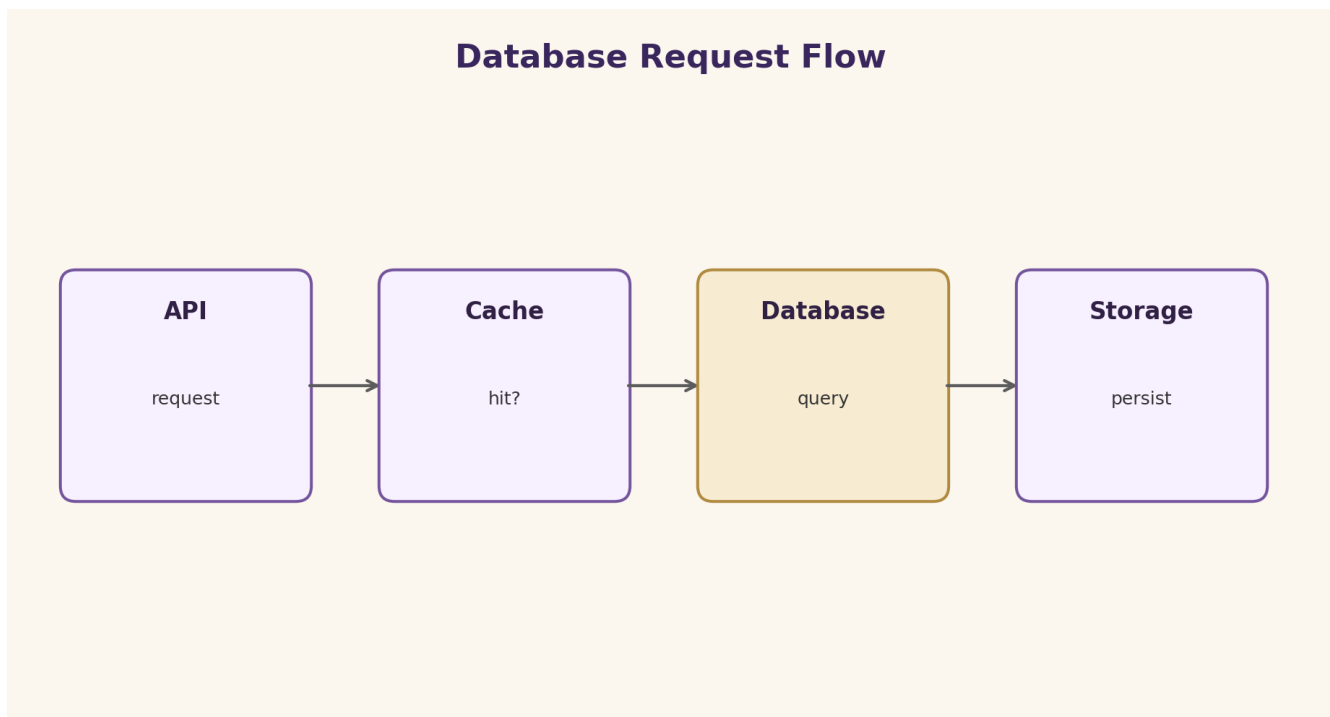
A structured answer

11. Relationships — If the domain has many strong relationships and the application needs joins, SQL's relational model is convenient.
12. Consistency — If correctness depends on multi-step transactions and strong consistency, a transactional SQL system may be a natural fit.
13. Access patterns — If queries are varied, relational databases provide expressive querying; if access patterns are narrow and predictable, a NoSQL model may be more directly optimized.
14. Scale — SQL can scale substantially, but the scaling approach depends on the engine and workload. Some NoSQL systems are designed around partitioning and distributed scale from the start.
15. Operational trade-offs — Consider team expertise, tooling, backup/restore, monitoring, replication, migrations, cost, failure modes, and the specific database's operational complexity.
16. Conclusion — Choose the database whose data model and guarantees best match the application's requirements.

SAMPLE INTERVIEW RESPONSE

“I would choose SQL when the system has strong relationships, transactional workflows, consistency requirements, and complex or changing queries. I would consider NoSQL when the access patterns are well known and the chosen NoSQL model gives clear benefits in flexible data modeling, distributed scale, or throughput. I would also compare operational complexity, team expertise, replication, backup, cost, and failure handling before deciding.”

15. How Databases Fit Into a Real System



- API layer receives the request.
- Cache may satisfy frequently repeated reads.

- Database performs the authoritative read/write.
- Indexes and query design determine how efficiently the database finds data.
- Transactions and isolation protect important multi-step operations.
- Replication and scaling strategies help the database handle larger workloads.
- Application architecture should respect the database's consistency and failure behavior.

16. 🗄️ One-Page Interview Cheat Sheet

Concept	Remember this
Table	Structured collection of rows and columns.
Primary Key	Uniquely identifies a row.
Foreign Key	References a related row in another table.
Index	Extra structure that can speed reads but adds storage/write cost.
JOIN	Combines related rows across tables.
Transaction	Logical unit of database work.
ACID	Atomicity, Consistency, Isolation, Durability.
Isolation	Controls visibility/interference between concurrent transactions.
Lock	Coordinates conflicting concurrent operations.
Query Optimization	Reduce unnecessary work; inspect execution plans and measure.
Document Store	Stores document-shaped records.
Key-Value	Key → value access model.
Wide-Column	Distributed model built around partitioned rows/columns.
Access Pattern	How the application actually reads/writes data.
Consistency Trade-off	Balance correctness/visibility, latency, availability, and distribution.
SQL vs NoSQL	Choose based on relationships, consistency, access patterns, scale, and operations.

17. 🎯 Interview Practice Questions

17. Why can an index make a query faster? What are the downsides?

18. Primary key vs foreign key — what problem does each solve?
19. INNER JOIN vs LEFT JOIN — when would you use each?
20. What is a transaction? Give a real example.
21. Explain ACID using a bank transfer.
22. What is a dirty read?
23. What causes a deadlock?
24. How would you investigate a slow SQL query?
25. When would a document database be useful?
26. Why is access-pattern-first design important in NoSQL?
27. Explain strong vs eventual consistency.
28. Why might denormalization improve performance?
29. Why is “NoSQL is always faster” an incorrect statement?
30. How would you decide between SQL and NoSQL for a new application?

MASTER THE WHY

For system design interviews, syntax is only the starting point. Be able to explain why a database structure, index, transaction model, consistency level, or NoSQL model fits the requirements.

PHASE 3 COMPLETE ✓

Requirements → Data Model → Queries → Transactions → Concurrency → Scale → Trade-offs